M4：文件、命令和 Git 元数据

七个工具的精确合同与工作区并发设计

M4：文件、命令和 Git 元数据
- 背景、目标与范围
- 公共行为与路径规格
  - M4-S01：路径解析
  - M4-S02：工具结果
  - M4-S03：统一截断
- 七个工具的操作规格
  - read
  - write
  - edit
  - grep
  - find
  - ls
  - bash
- Git 元数据规格
  - M4-S04：仓库发现
  - M4-S05：分支显示
- 数据结构与设计模式
  - Command 与策略
  - 按工作区串行执行器
  - 原子替换与读写冲突
  - 端口与适配器
- 验收示例与矩阵
- 工具规格补充示例
  - 行号和截断
  - 搜索输出路径
  - 写队列失败恢复
  - 错误优先级与副作用
- 取舍与交付顺序

M4：文件、命令和 Git 元数据

属性	内容
状态	设计提案 1.0
前置	M1 工具合同、M3 资源允许列表
交付	`@local/tools`，固定顺序 read/write/edit/grep/find/ls/bash
验收	M4-A01 至 M4-A14

背景、目标与范围

编程助手需要观察和修改工作目录。文件工具应明确读多少、改什么和失败后保留什么；命令执行还需要处理超时、输出过大和子进程。Git 元数据用于界面展示，不是另一套提交系统。

目标是可组合的 AgentTool 实现，精确编辑、稳定搜索、UTF-8 安全截断和同工作区写入串行。非目标是完整 shell 沙箱、模糊补丁、自动 commit/worktree、搜索所有编码文件和全量 Git 实现。模型若需要 Git 操作，通过 bash 请求，程序不会隐式提交。

公共行为与路径规格

M4-S01：路径解析

`createDefaultTools({workspace,cwd,readAllowlist})` 捕获 realpath workspace/cwd，验证 cwd 在 workspace 内。传入工具的相对路径相对 cwd；允许 workspace 内的绝对路径。路径判断用 `path.relative(root,target)`，拒绝结果为 `..`、以 `..` 路径段开头或绝对路径，不用字符串 startsWith 代替目录关系。

基线对用户给定的每个路径段执行 lstat，拒绝符号链接，防止跟随出工作区；不能只 resolve 文本中的 `..`。write 的不存在后缀逐级检查已有父目录，创建父目录后再检查。路径 NUL、非字符串、空字符串为 INVALID_ARGS。read 的精确 allowlist 是唯一范围例外：必须是已扫描、普通、非符号链接的完整文件路径，不能传其父目录读取其他文件。

所有非 allowlist 文件操作限于 workspace，包括 grep/find/ls。遍历不跟随 link。工作区由可信本地用户控制；这些检查不防止恶意并发进程在校验后替换目录（TOCTOU），也不隔离 bash。需要敌对环境时须另做 OS 沙箱设计。

M4-S02：工具结果

工具返回 `{content,isError?,error?,details?}`，均满足 M1 ToolOutcome。content 给模型看，details 给 UI 和测试，不复制全部大文件。预校验失败直接抛带 code 的领域错误，由 Core 归一化；命令非零退出可直接返回 isError=true，以保留退出码和输出。

读类 effect=read；write/edit=write；bash=external。M1 看到任何 write/external 会串行整批。write/edit 仍共享工作区 mutation queue，以防不同调用入口并发进入同一进程。

M4-S03：统一截断

所有文本型工具输出最多 2000 行、51200 UTF-8 字节，先到上限停止。超限时在内容末尾附 `\n[truncated]\n`，标记也算在字节预算内；调用者可降低上限，但必须足够容纳标记。按完整 Unicode 码点截取，不产生 replacement character。CRLF 归一成 LF 只用于展示，不改原文件。

逻辑行定义为按 LF 分隔，尾部 LF 不再制造一个空行，空文本为 0 行。超长单行可在码点边界截字节，保留不完整行并标 truncated。`details` 含 originalBytes/returnedBytes/truncated；流式命令只保证 observedBytes 和 retainedBytes，不为了统计总行数保存全部输出。

read/grep/find/ls 保留头部；bash 使用有界尾部环形缓冲保留最新输出，因为报错常在尾部。bash 截断标记在开头。onUpdate 也必须有界，不能先把无限输出塞进内存再截。

七个工具的操作规格

read

参数：`{path:string,offset?:integer,limit?:integer}`，offset 为 1-based，默认 1；limit 默认 2000、合法 1..2000，未知字段拒绝。读取普通 UTF-8 文本文件；NUL 或无效 UTF-8 视为非文本，返回 INVALID_INPUT，不做图片转换。

效果：返回从 offset 开始最多 limit 行，每行前缀 `<行号>:`，然后应用字节上限。details=`{path,startLine,endLine,totalLines,truncated,nextOffset?}`，path 是绝对路径。EOF 之前不足 limit 行不补空行；nextOffset 是下一条尚未返回的完整源行。如果字节上限截断了某行，nextOffset 指向该行，调用者可缩小请求范围；不声称该行完整返回。

边界：空文件返回空文本、startLine=1、endLine=0、totalLines=0。offset>totalLines 对非空文件返回空内容和 endLine=offset-1；不是 NOT_FOUND。offset<1、limit<=0 为 INVALID_ARGS。不存在 NOT_FOUND，目录 INVALID_INPUT，越界 PATH_ESCAPE。read 无磁盘变更。

成本：扫描到需要统计的 EOF 为 O(文件字节数)，内存有界于输出+单个解码块。不要用 split 载入巨大文件；单行也应流式限制。readAllowlist 仅在 read 生效。

write

参数：`{path:string,content:string}`，空 content 合法，UTF-8 编码。效果：全量替换或创建，父目录可递归创建，返回 `Wrote <bytes> bytes.`，details=`{path,bytes,created}`。

在共享 mutation queue 内检查路径，创建同目录随机临时文件，以新文件 0600 或原文件权限写入、同步、关闭，再 rename 覆盖。已有文件为目录/符号链接拒绝；不因旧内容碰巧相同而改变返回规则。失败清理临时文件，rename 前保持旧目标字节不变；可能留下新建父目录，不保证目录事务回滚。取消在 rename 前检查，取消后删除临时文件；rename 已成功则按成功上报，不能谎称未写。

成本：O(content 字节数) 时间和临时磁盘空间，额外内存不超输入字符串和写缓冲。写不在 workspace 的 allowlist 文件仍 PATH_ESCAPE。

edit

参数：`{path:string,oldText:string,newText:string}`。oldText 非空，newText 可空。只修改已有普通 UTF-8 文件，不创建文件。精确匹配源文件字节解码后的字符串，不规范化 CRLF、缩进或 Unicode。

统计所有匹配起点，包括重叠匹配；必须恰好 1 个。零次或多次为 CONFLICT，details 带 occurrenceCount，文件原字节不变。替换一次后按 write 的原子临时文件方案提交，保留现有权限。oldText===newText 但唯一匹配仍成功，replacements=1、changed=false。

成功 content 为 `Replaced one occurrence.`，details=`{path,replacements:1,changed,bytes}`。文件不存在 NOT_FOUND，内容无效 INVALID_INPUT。读→计数→替换→提交必须在同一 mutation queue 工作项内，不能在入队前读取。

例：`aaaa` 中 oldText=`aa` 有 3 个重叠起点，拒绝；`hello\nworld` 中 oldText=`world` 唯一，可换成 `agent`。成本 O(n+m) 可用 KMP；内置逐次 indexOf 也可，但不得在大量重叠文本上反复复制后缀造成二次方内存。

grep

参数：`{pattern:string,path?:string}`，path 默认 cwd，可以是文件或目录。pattern 是 JavaScript Unicode 正则源，逐行匹配，不跨行，不加 g/i。非法正则 INVALID_ARGS。每行只输出一次，格式 `<相对cwd路径>:<1-based行号>:<原行>`。

递归枚举普通文件，相对路径码点排序，再按行号递增输出。跳过 `.git`、`node_modules`、符号链接、二进制和不可读文件；后两类在 details 计数，不因单文件失败抛弃全部结果。用户指定的起始 path 不可读则 TOOL_FAILED，不存在 NOT_FOUND。无匹配是成功空文本。

details=`{matches,filesScanned,filesSkipped,truncated}`，matches 是已观察到的匹配行数；一旦达到输出/扫描预算可停止，不能称其为工作区总匹配数。扫描预算最多 10000 文件和 100 MiB 文本，超限 truncated=true。复杂度为 O(扫描字节数×正则代价 + 路径排序)，JS 正则可能灾难性回溯；基线仅用于可信本地 pattern，不承诺任意恶意模式可中断。

find

参数：`{pattern:string,path?:string}`。pattern 是大小写敏感的 basename 子串，不是 glob/正则，空字符串表示所有普通文件。起始目录默认 cwd，必须为目录。遍历规则和扫描文件数量上限同 grep，但不读文件内容。

输出相对 cwd 的 POSIX 风格路径，一行一个，按码点排序。只输出文件，不输出目录或 link；无匹配成功。details=`{matches,truncated}`。起始文件为 INVALID_INPUT，目录不存在 NOT_FOUND。时间 O(f log f)，存储路径 O(f)；f 受扫描上限约束。

ls

参数：`{path?:string}`，默认 cwd。只列一层，目录 basename 后加 `/`，普通文件原名，符号链接加 `@` 但不跟随。不隐藏点文件，排序按不带后缀的名字码点顺序。

输出一行一个名称；空目录成功空文本。details=`{entries,truncated}`。path 非目录 INVALID_INPUT，不存在 NOT_FOUND。O(k log k)，k 为直接子项数；不读取文件内容。

bash

参数：`{command:string,timeoutMs?:integer}`，command 非空，timeout 默认 120000，合法 1..120000。固定使用 POSIX `bash -lc <command>`，cwd 为构造时 cwd；不把用户文本再拼接到其他 shell 片段。环境继承进程环境，所以 bash 不是工作区沙箱。Windows 不在基线，启动时明确 INVALID_CONFIG，不错误套用 Bash 参数调用 PowerShell。

stdout/stderr 按接收 chunk 的次序合并，说明这不是两个 fd 的严格因果总序。流式 UTF-8 解码，尾部缓冲有界，每至多 100ms 发一次 update，最终结果立即发。exit=0 返回成功；非零返回 TOOL_FAILED 且保留退出码和尾部输出；进程被信号终止保存 signal、exitCode=null。

超时或取消先向独立进程组发 SIGTERM，等待 1000ms，再 SIGKILL，最后等待 close。没有 close 不得返回“已清理”。
details=`{exitCode,signal,timedOut,aborted,observedBytes,truncated}`；timeout 返回 TIMEOUT，取消返回 ABORTED。spawn 失败 TOOL_FAILED，不编造 exitCode=0。测试用子进程和假计时，不运行会产生外部变更的命令。

Git 元数据规格

M4-S04：仓库发现

`findGitPaths(cwd)` 向父查找第一个 `.git`，直到文件系统根。仅供元数据观察；不扩展工具 workspace 权限。无 `.git` 返回 null。遇到 `.git` 但损坏返回带 diagnostic 的 null，不继续误认更上层仓库。

.git 形态	解析
目录	gitDir=.git，要求 HEAD 存在，commonGitDir=gitDir

.git 形态	解析
文件	首行匹配 `gitdir: <path>`，相对该文件所在目录解析 gitDir，要求 HEAD 存在
gitDir 有 commondir 文件	相对 gitDir 解析 commonGitDir，校验目录存在

返回 `{repoDir,gitDir,commonGitDir,headPath}`，headPath=gitDir/HEAD。普通仓库与 linked worktree 都正确；Git 管理路径可在 workspace 外，仅供 metadata 读取，不加入 readAllowlist。

M4-S05：分支显示

`resolveBranch(repoDir)` 用参数数组运行 `git --no-optional-locks symbolic-ref --quiet --short HEAD`，超时 1000ms。成功非空 stdout → `{kind:'branch',name}`；退出 1 → `{kind:'detached'}`；无 Git/其他退出/超时 → `{kind:'unavailable'}` 并诊断；不能把所有错误显示为 detached。

不通过读 HEAD 文本猜测分支，不调用 status、fetch 或任何写命令。M6 在 idle/turn_end 后最多每 500ms 刷新一次，命令不占模型上下文。worktree 的会话文件按自身 realpath workspace hash 分开，不与主工作树共享锁或上下文。

数据结构与设计模式

```
type FileToolOptions = {
  workspace: string;
  cwd: string;
  readAllowlist: ReadonlySet<string>;
};
type Truncation = {
  content: string;
  truncated: boolean;
  originalBytes: number;
  returnedBytes: number;
};
interface MutationQueue {
  run<T>(work: () => Promise<T>, signal?: AbortSignal): Promise<T>;
}
type GitPaths = {
  repoDir: string;
  gitDir: string;
  commonGitDir: string;
  headPath: string;
};
type BranchState =
  | { kind: "branch"; name: string }
  | { kind: "detached" }
  | { kind: "unavailable" };
```

Command 与策略

每个 AgentTool 是一个 Command：schema 是输入合同，execute 是操作，结果用共同结构返回。read/write 的路径规则共享 PathPolicy；头/尾截断是两种 Strategy，不各写一份字节切割。工具不注册 UI，也不写会话。

按工作区串行执行器

使用 `Map<realWorkspace,queue>`，同一进程对同一 workspace 获取同一队列，write/edit 都在内部调用它。队列 tail 是 Promise，新的 work 挂在已结算 tail 后；前任务失败仍释放下一任务。等候中的 signal 已取消则跳过，不执行 work。finally 回收 idle 队列引用，防止长期 Map 泄漏。

这保证进程内同工作区 write/edit 不重叠，不保证外部编辑器或 bash 不改文件。M1 整批串行避免同批 bash 和文件操作竞争；不同进程之间没有分布式文件事务。

原子替换与读写冲突

同目录临时文件 + rename 能避免本进程主动留下半文件；不是磁盘级事务保证。若要防止外部编辑器覆盖，可后续加入 expectedHash 乐观并发字段，但不属于本基线。当前 edit 在读取至提交之间被外部修改可能覆盖，文档必须保留这一限制。

端口与适配器

GitExec、ProcessRunner 和 FileSystem 边界可注入，用于制造 timeout、EACCES 和短写。不要 mock 核心字符串替换函数来证明 edit 正确；应用临时文件验证最终字节。进程组清理需至少一条真实 POSIX 子进程测试，假 runner 不能证明 OS 行为。

验收示例与矩阵

编号	输入或场景	必须断言
M4-A01	`../outside`、同前缀 sibling、目录 symlink	PATH_ESCAPE；不改外部文件
M4-A02	allowlist 外部技能文件	read 可读；write/edit/ls 不获权限
M4-A03	read 多行 offset=2,limit=1；空文件；越 EOF	行号、endLine、nextOffset 与规格一致
M4-A04	中文/emoji 恰在字节界；超长单行	有效 UTF-8；预算含标记；无半字符
M4-A05	write 覆盖/创建/空内容，rename 前注入失败	成功字节准确；失败原目标不变；临时文件清理
M4-A06	edit 0/1/多/重叠匹配，CRLF	仅唯一成功；不自动规范化；其他原字节不变
M4-A07	同 workspace 两次创建工具表后并发 edit/write	共用队列；前失败不阻塞后任务

编号	输入或场景	必须断言
M4-A08	grep 匹配/无匹配/非法正则/二进制	稳定顺序；无匹配成功；跳过有计数
M4-A09	find 子串与 `*.ts`；ls 隐藏/link/空目录	不把 find 当 glob；ls 不递归
M4-A10	bash 输出中文分片和大量输出，exit 0/3	解码正确；内存有界；非零保留输出和 code
M4-A11	bash 启动一个长寿命子进程后 abort/timeout	进程组收到终止；等待 close；ABORTED/TIMEOUT
M4-A12	普通仓库、相对 gitdir、commondir worktree	四个路径正确；坏标记不向上查找
M4-A13	branch/detached/无 Git	三种显示态明确；GitExec 参数无写操作
M4-A14	端到端 read→edit→read→最终答复	新文件内容、工具结果和持久化 callId 一致

测试超时必须有独立 watchdog 清理夹具进程。测试路径带空格，确保参数数组和路径处理不依赖 shell 自动转义。

工具规格补充示例

行号和截断

文件字节为`a\nb\n`，共有2行。read(offset=2,limit=1)返回`2: b`，endLine=2，无nextOffset；read(offset=3)返回空，endLine=2。空文件不应该显示`1:`伪造一行。

行数预算包含截断提示行；发生截断时先给提示保留一行和对应字节，再选择内容。UTF-8计数发生在行号等展示前缀加入之后，而details.originalBytes指原始选区字节，两者不能混淆。read因单行过长截断可能无法靠nextOffset翻页获取整行，这是基线限制；可用bash等其他显式操作处理，不默默把offset改成字节偏移。

搜索输出路径

cwd=`/w/src`、workspace=`/w`时，匹配`/w/test/a.ts`输出`../test/a.ts:3: ...`合法，因为访问仍在workspace内。相对展示路径含`..`不等于路径逃逸，访问校验必须基于workspace；若把cwd误当访问边界，会拒绝合法兄弟目录。

文件名可能包含换行或控制字符，搜索输出用JSON字符串转义路径部分，details始终保存原路径；内容交TUI再次sanitize。普通不含特殊字符路径保持可读文本，不直接在终端写原始ESC。

写队列失败恢复

设Q1=edit、Q2=write，Q1因匹配两次CONFLICT。Q2仍必须执行，不能把tail永久保存为rejected Promise。若Q2在等待时已取消，跳过work；若Q2已经rename成功才收到取消，上报成功并保留新字节。

队列只覆盖宿主管理的write/edit。若用户在外部编辑器同时修改文件，本方案没有expectedHash就不能承诺不覆盖。不能因采用临时文件rename就宣称有并发冲突检测。

错误优先级与副作用

操作阶段	失败	行为
参数schema	limit为字符串、未知键	INVALID_ARGS，零文件I/O
路径范围	指向外部或经过link	PATH_ESCAPE，目标不打开
起始目标	不存在/类型错误	NOT_FOUND/INVALID_INPUT
edit计数	0或多个匹配	CONFLICT，原字节不变
临时文件写入	短写后故障	TOOL_FAILED，清临时文件，原文件不变
rename完成后记录结果失败	PERSISTENCE发生在Session	文件已改变，恢复不重试
bash非零	exitCode=3且有stderr	TOOL_FAILED，保留尾部输出，不能回滚命令

write/edit内部不能catch后返回“修改成功”，否则Core与会话会永久记录错误事实。工具输出的错误细节不得泄漏密钥环境变量；bash本身执行的用户命令若主动打印秘密仍会进入会话，这是操作选择的结果，需要用户自行控制命令内容。

取舍与交付顺序

先做 PathPolicy、UTF-8 截断和队列，再做 read/write/edit，之后搜索和 bash，最后 Git 元数据。搜索用 Node API 实现基础版，避免外部 ripgrep 依赖；可在保持排序、错误和预算语义后替换执行器。

保守拒绝 symlink 限制了部分合法项目布局，换来可解释的基线。bash 保留完整 shell 能力，所以不能宣传“所有工具都只能操作工作区”。自动 Git commit 可能掩盖用户未确认的修改，留给显式任务指令通过 bash 完成。